Double Machine Learning Simulation and Extension Proposal for More General Treatment Effects

Jihye Lee
jlee92603@ucla.edu

March 14, 2026

Abstract

Double machine learning (DML) provides a robust framework for estimating causal parameters, such as treatment effects, while accounting for high-dimensional confounding variables through nuisance function estimation. This paper evaluates the performance of the DML estimator using various machine learning models such as Ordinary Least Squares (OLS), LASSO, Random Forest (RF), and Neural Networks (NN) within a simulated environment where the true treatment parameter is known. Our findings indicate that Random Forest yielded the most accurate estimates of the target parameter in this specific simulation. Furthermore, we propose a methodological extension to the DML framework using a Taylor series expansion to estimate non-linear treatment effect functions, allowing for greater flexibility in modeling complex causal relationships.
Keywords: *Double Machine Learning; Taylor Series; DML.*

1 Introduction

Double machine learning (DML) is an approach to perform inference about a target parameter when there are nuisance functions. Nuisance functions are objects needed to identify the target parameter but are not of primary interests. They are also known as covariates or confounders. The goal of double machine learning is to estimate and construct confidence intervals for low-dimensional parameter (θ_0) in the presence of high-dimensional nuisance parameter (η_0). The nuisance parameter may be estimated with nonparametric statistical methods such as random forests, boosted trees, lasso, ridge, neural networks, and more. This paper designs a simulation study to compare the performance of the DML estimator with ordinary least squares and investigates how the choice of machine learning method used to estimate the nuisance function $g_0(X)$ affects the estimation accuracy of θ_0 as well as proposes an extention of the DML framework for estimating a more general treatment effect function $f_0(D)$ when the treatment effect is not linear in D.

2 Methodology

2.1 Comparison of Various Methods

Double machine learning framework is proposed by the partially linear model

$$Y = D\theta_0 + g_0(X) + \varepsilon$$

$$E[\varepsilon|X, D] = 0$$

where D is the treatment variable, X is a vector of covariates, and θ_0 is the parameter of interest (Ahrens et al. (2026)). $g_0(X)$ is the non linear nuisance function. It makes the relationship between the covariates and the outcome complex. A number of folds of 5 was used for cross-fitting to prevent overfitting.

In the simulation study, 1000 observations for 50 features are used. The true parameter of interest is 5. 3 different machine learning methods are selected for the double machine learning: LASSO, random forest (rf), and a multi-layer perceptron neural network (nn). These methods are compared against the baseline ordinary least squares regression (ols).

The simulation is ran 50 times.

2.2 Proposed Extension

To estimate a more general treatment effect function $f_0(D)$ when the treatment effect is not linear in D, using Taylor series expansion to linearize nonlinear functions by approximating them as linear equation is proposed (Colangelo and Lee (2023)).

$$f_0(D) = f(0) + \frac{f'(0)}{1!}D + \frac{f''(0)}{2!}D^2 + ... = \sum_{n=0}^{\infty} \frac{f^{(n)}(0)}{n!}D^n$$

A Maclaurin series is used with the sum of derivatives at 0. The DML estimator treats each derivative as unknown coefficient that it needs to learn from the data. Each power is a separate treatment.

$$Y = \theta_1 D + \theta_1 D^2 + \theta_3 D^3 + \theta_4 D^4 + \theta_5 D^5 + g_0(X) + \varepsilon$$

Using a fifth-degree polynomial allows for a bias-variance tradeoff and taylor approximation theorem. A fifth-degree polynomial captures the essence of most smooth functions. A 4th degree easily captures the initial wave of of sine and cosine functions. 4th degree polynomials can also mimic the shape of log curves over a finite range. Adding more orders increases the variance of my estimates and the model may being to capture noise and overfit rather than capturing the true signal. By stopping at the fifth degree, we balance the approximation bias with the estimation variance.

3 Results

3.1 Results for Part 1

After running various DML methods on the simulated data, we found that to estimate the true parameter, Random Forest performs most accurately. Then LASSO and OLS, and then the neural network. For a true parameter theta of 5.0, the mean ordinary least squares parameter estimate was 5.5745, the mean LASSO parameter estimate was 5.5787, the mean random forest estimate was 5.0570, and the mean neural network parameter estimate was 4.1472 as written in table 1, under Supplemental Materials. The difference in performances can be seen in figure 1 and figure 2, under Supplemental Materials.

3.2 Results for Part 2

The proposed extension of using a fifth-degree Taylor Series expansion for DML reveals a trade-off between local accuracy and global approximation bias. As illustrated in Figure 4, under Supplemental Materials, the estimator successfully captures the "signal" or the shape of the non-linear treatment effect $f_0(D)$ for values of D near the expansion point. For instance, in the quadratic and cubic function simulations, the estimated curve closely tracks the true signal within the interval $D \in [-4, 0]$.

However, the performance degrades significantly as D moves further from the origin. The Taylor approximation begins to diverge, failing to capture the steeper gradients or high-frequency oscillations later in the range. This suggests that while the fifth-degree polynomial is sufficient for capturing simpler curvature, it is susceptible to approximation error when the underlying treatment effect is highly non-linear or spans a wide domain.

4 Discussion and Conclusions

In this simulation, the Random Forest (RF) estimator provided the most accurate mean estimate (5.057) compared to the true parameter of 5.0. In contrast, OLS (5.574) and LASSO (5.578) tended to overestimate the parameter, while the Neural Network (4.147) significantly underestimated it.

The performance of Random Forest in this context likely stems from its ability to capture complex, non-linear interactions between covariates without requiring explicit functional form specifications. While Neural Networks are theoretically capable of modeling any continuous function, they often require more data and more extensive hyperparameter tuning (e.g., layers, neurons, learning rate) than what was utilized in this simulation. The bias observed in the OLS and LASSO estimates suggests that the linear assumptions underlying those models may not have fully captured the nuisance relationships $g_0(X)$ present in the data generating process.

This study highlights the importance of the choice of machine learning method within the Double Machine Learning framework. Through simulation, we demonstrated that Random Forest can be particularly effective at estimating the nuisance effects. Additionally, we proposed an extension of the DML framework using a fifth-degree Taylor polynomial approximation to estimate non-linear treatment effects. This approach allows to maintain to reduce the nuisance functions while expanding its applicability to more general treatment effect functions $f_0(D)$.

References

Ahrens, A., Chernozhukov, V., Hansen, C., Kozbur, D., Schaffer, M., & Wiemann, T. (2026). *An introduction to double/debiased machine learning.* Retrieved from https://arxiv.org/abs/2504.08324

Colangelo, K., & Lee, Y.-Y. (2023). *Double debiased machine learning nonparametric inference with continuous treatments.* Retrieved from https://arxiv.org/abs/2004.03036

Supplemental Materials

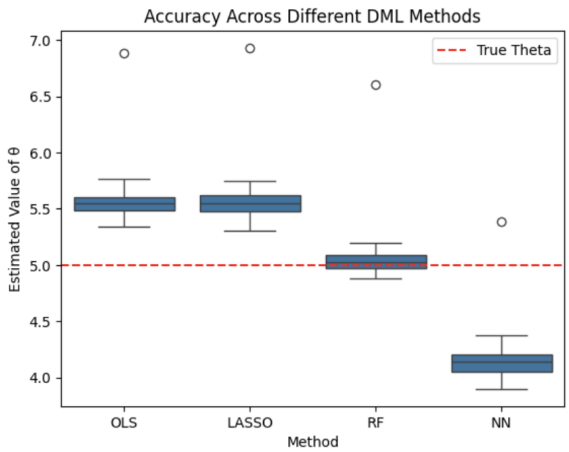
4.1 Code Availability

The code is available at https://github.com/jlee92603/double_machine_learning/

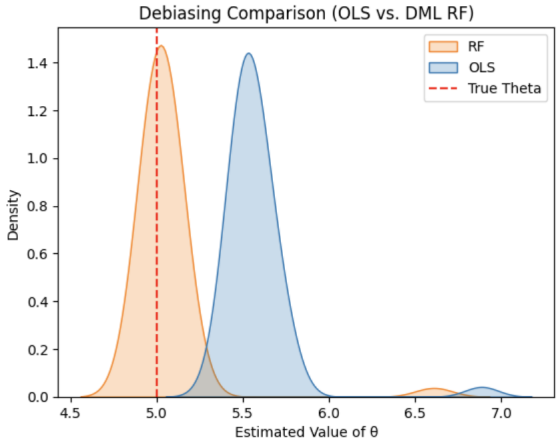
4.2 Tables and Figures

Summary of the Estimated Parameter for the Simulation by Each Method

Method	Mean	St. Dev.	Min	25%	50 %	75 %	Max
Ordinary Least Squares	5.574455	0.211764	5.342781	5.481273	5.544687	5.605578	6.885543
LASSO	5.578704	0.219991	5.309066	5.478200	5.545956	5.616739	6.929687
Random Forest	5.057049	0.236021	4.882593	4.975717	5.022557	5.089444	6.608447
Neural Network	4.147182	0.209408	3.898874	4.051402	4.142875	4.202135	5.388403



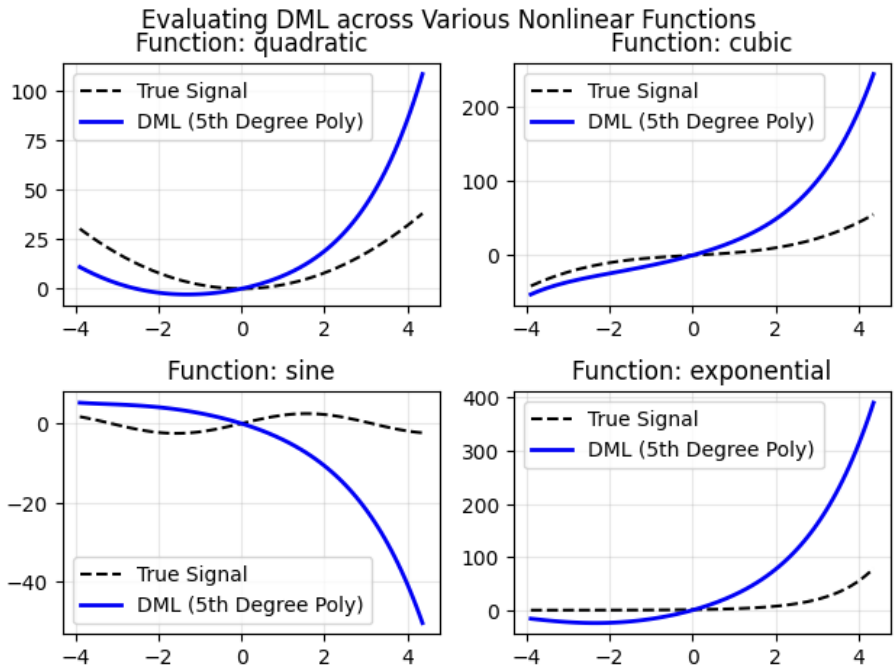
Accuracy of Parameter Estimates Across Various Methods



Comparison between the parameter estimation between OLS and DML RF for the 50 simulation iterations

```
--- Results for QUADRATIC ---
True Equation:      f(D) = 2*D^2
Estimated Equation: f_hat(D) = 4.455*D + 1.811*D^2 + 0.170*D^3 + 0.078*D^4  0.008*D^5

--- Results for CUBIC ---
True Equation:      f(D) = 0.5*D^3 + 3*D
Estimated Equation: f_hat(D) = 15.564*D + 2.558*D^2 + 0.541*D^3 + 0.109*D^4  0.026*D^5

--- Results for SINE ---
True Equation:      f(D) = 2.5*sin(D)
Estimated Equation: f_hat(D) = -3.257*D + -0.728*D^2 + -0.097*D^3 + -0.024*D^4  -0.004*D^5

--- Results for EXPONENTIAL ---
True Equation:      f(D) = exp(D)
Estimated Equation: f_hat(D) = 21.715*D + 5.734*D^2 + 0.734*D^3 + 0.220*D^4  0.029*D^5
```

The true and estimated non-linear treatment equations



The true and estimated signals for various non-linear treatment equations plotted

4.3 Code

[4]
13s

```
pip install doubleml
```

Show hidden output

[5]
1s

```
### IMPORTS ###
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import statsmodels.api as sm
from doubleml import DoubleMLData, DoubleMLPLR

from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LassoCV, LinearRegression
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import PolynomialFeatures
```

[28]

```
# parameters
np.random.seed(67)
n = 1000 # number of observations
p = 50 # number of variables
theta_true = 5.0

# models
lasso = LassoCV(cv=5)
rf = RandomForestRegressor(n_estimators=100, max_depth=5)
nn = MLPRegressor(hidden_layer_sizes=(100, 50), max_iter=1000)

# run models in iteration
n_iter = 50
results = []

for i in range(n_iter):

  X = np.random.normal(0, 1, (n,p))
  g0 = np.sin(X[:, 0]) + np.exp(X[:, 1])
  m0 = X[:, 0] + 0.5 * (X[:, 1]**2)
  D = m0 + np.random.normal(0, 1, n)
  Y = D * theta_true + g0 + np.random.normal(0, 1, n)

  df = pd.DataFrame(X, columns=[f'X{j}' for j in range(p)])
  df['D'], df['Y'] = D, Y
  obj_dml_data = DoubleMLData(df, 'Y', 'D')

  # baseline ordinary least squares
  ols = sm.add_constant(df.drop(columns=['Y']))
  ols_model = sm.OLS(df['Y'], ols).fit().params['D']
  results.append({'Method': 'OLS', 'Estimate':ols_model})

  # LASSO
  dml_lasso = DoubleMLPLR(obj_dml_data, ml_l=lasso, ml_m=lasso, n_folds=5)
  dml_lasso.fit()
  results.append({'Method': 'LASSO', 'Estimate':dml_lasso.coef[0]})

  # Random Forest
  dml_rf = DoubleMLPLR(obj_dml_data, ml_l=rf, ml_m=rf, n_folds=5)
  dml_rf.fit()
  results.append({'Method': 'RF', 'Estimate':dml_rf.coef[0]})

  # Neural Network
  dml_nn = DoubleMLPLR(obj_dml_data, ml_l=nn, ml_m=nn, n_folds=5)
  dml_nn.fit()
  results.append({'Method': 'NN', 'Estimate':dml_nn.coef[0]})

results_df = pd.DataFrame(results)
```

[30]

```python
# SAVE / LOAD Results
results_df.to_csv('stat201b_final_results.csv', index=False)
results_df = pd.read_csv('stat201b_final_results.csv')
```

[69]

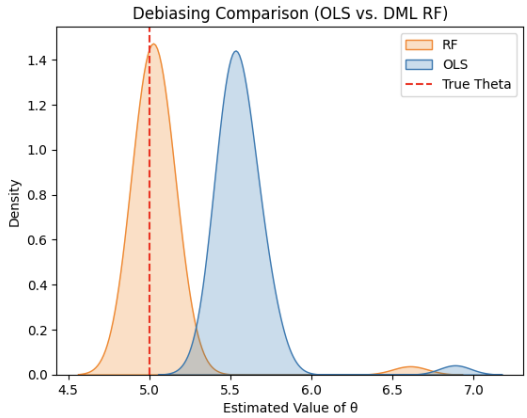
```python
print('theta true: ', theta_true)
results_df.groupby('Method')['Estimate'].describe()
```

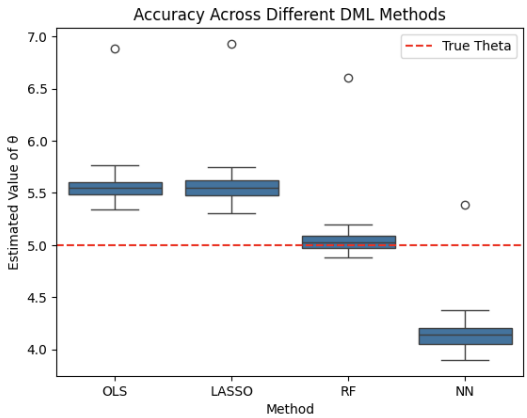
```
theta true:  5.0
```

Method	count	mean	std	min	25%	50%	75%	max
LASSO	50.0	5.578704	0.219991	5.309066	5.478200	5.545956	5.616739	6.929687
NN	50.0	4.147182	0.209408	3.898874	4.051402	4.142875	4.202135	5.388403
OLS	50.0	5.574455	0.211764	5.342781	5.481273	5.544687	5.605578	6.885543
RF	50.0	5.057049	0.236021	4.882593	4.975717	5.022557	5.089444	6.608447

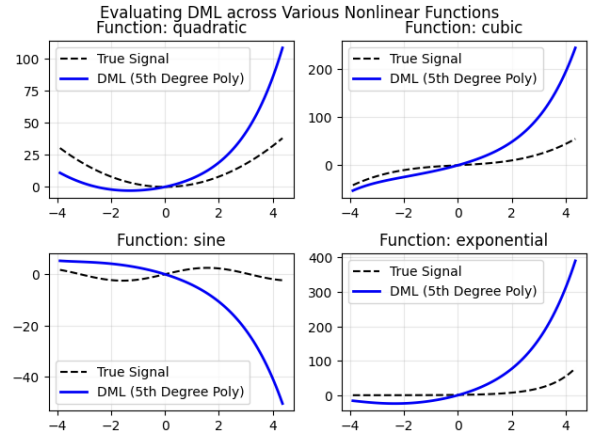
[54]

```python
# comparison of debiasing
plt.figure()
subset = results_df[results_df['Method'].isin(['OLS', 'RF'])]
sns.kdeplot(data=subset, x='Estimate', hue='Method', fill=True)
plt.axvline(theta_true, color='red', linestyle='--', label='True Theta')
plt.title('Debiasing Comparison (OLS vs. DML RF)')
plt.xlabel('Estimated Value of θ')
plt.legend(['RF', 'OLS', 'True Theta'])
plt.show()

# performance comparison
plt.figure()
sns.boxplot(data=results_df, x='Method', y='Estimate')
plt.axhline(theta_true, color='red', linestyle='--', label='True Theta')
plt.title('Accuracy Across Different DML Methods')
plt.xlabel('Method')
plt.ylabel('Estimated Value of θ')
plt.legend()
plt.show()
```





[73]

```python
# parameter set up
np.random.seed(67)
n = 1000
p = 50
X = np.random.normal(0, 1, (n, p))
g0 = np.sin(X[:, 0]) + np.exp(X[:, 1])
D = X[:, 0] + np.random.normal(0, 1, n)

# non linear treatment equations
eqns = {
    "quadratic": lambda d: 2 * (d**2),
    "cubic": lambda d: 0.5 * (d**3) + 3 * d,
    "sine": lambda d: 2.5 * np.sin(d),
    "exponential": lambda d: np.exp(d),
}

true_eq_strings = {
    "quadratic": "f(D) = 2*D^2",
    "cubic": "f(D) = 0.5*D^3 + 3*D",
    "sine": "f(D) = 2.5*sin(D)",
    "exponential": "f(D) = exp(D)"
}

fig, axes = plt.subplots(2, 2)
axes = axes.flatten()

rf = RandomForestRegressor(n_estimators=100, max_depth=5)

for i, (name, eqn) in enumerate(eqns.items()):
    Y = eqn(D) + g0 + np.random.normal(0,1,n)

    poly = PolynomialFeatures(degree=5, include_bias=False)
    D_expanded = poly.fit_transform(D.reshape(-1, 1))
    x_cols = [f'X{j}' for j in range(p)]
    d_cols = [f'D_pow{j+1}' for j in range(5)]

    # data
    df = pd.DataFrame(X, columns=x_cols)
    df[d_cols], df['Y'] = D_expanded, Y

    # run model (dml with rf)
    obj_data = DoubleMLData(df, 'Y', d_cols, x_cols)
    dml_model = DoubleMLPLR(obj_data, ml_l=rf, ml_m=rf, n_folds=5)
    dml_model.fit()

    # print equations
    coeffs = dml_model.coef
    est_eq = f"f_hat(D) = {coeffs[0]:.3f}*D + {coeffs[1]:.3f}*D^2 + {coeffs[2]:.3f}*D^3 + {coeffs[3]:.3f}*D^4  {coeffs[4]:.3f}*D

    print(f"\n--- Results for {name.upper()} ---")
    print(f"True Equation:      {true_eq_strings[name]}")
    print(f"Estimated Equation: {est_eq}")

    # visual comparisons
    D_grid = np.linspace(D.min(), D.max(), 100)
    f0_true = eqn(D_grid)

    # reconstruct estimated f(D)
    coeffs = dml_model.coef
    f0_hat = np.zeros_like(D_grid)
    for j in range(5):
        f0_hat += coeffs[j] * (D_grid**(j+1))

    # plot
    axes[i].plot(D_grid, f0_true, label='True Signal', color='black', linestyle='--')
    axes[i].plot(D_grid, f0_hat, label='DML (5th Degree Poly)', color='blue', linewidth=2)
    axes[i].set_title(f'Function: {name}')
    axes[i].legend()
    axes[i].grid(True, alpha=0.3)

plt.tight_layout()
plt.suptitle('Evaluating DML across Various Nonlinear Functions')
plt.subplots_adjust(top=0.90)
plt.show()
```

```
--- Results for QUADRATIC ---
True Equation:      f(D) = 2*D^2
Estimated Equation: f_hat(D) = 4.455*D + 1.811*D^2 + 0.170*D^3 + 0.078*D^4  0.008*D^5

--- Results for CUBIC ---
True Equation:      f(D) = 0.5*D^3 + 3*D
Estimated Equation: f_hat(D) = 15.564*D + 2.558*D^2 + 0.541*D^3 + 0.109*D^4  0.026*D^5

--- Results for SINE ---
True Equation:      f(D) = 2.5*sin(D)
Estimated Equation: f_hat(D) = -3.257*D + -0.728*D^2 + -0.097*D^3 + -0.024*D^4  -0.004*D^5

--- Results for EXPONENTIAL ---
True Equation:      f(D) = exp(D)
Estimated Equation: f_hat(D) = 21.715*D + 5.734*D^2 + 0.734*D^3 + 0.220*D^4  0.029*D^5
```